

1. *Cotação de cada pergunta:* 1. 30 / 2. 35 / 3. 35 (Total: 100 pontos)
2. Responda às questões de forma clara e concisa nas folhas de exame.

1. (Valorização: 30%) Responda às perguntas seguintes:

a) Faça o estudo da complexidade do seguinte método:

```
1. void f(int n) {  
2.     float delta, x;  
3.     for(int i = 1; i <= n; i++) {  
4.         delta = 1/i;  
5.         x = i;  
6.         while(x > 0)  
7.             x -= delta;  
8.     }  
9. }
```

b) Indique e justifique qual deve ser a complexidade assintótica de uma operação de inserção de N elementos numa heap, sabendo que cada elemento é adicionado no fim da heap aplicando-se-lhe depois o método `upHeap()` sobre esse elemento para se repor a condição de heap.

Nota: a resposta deverá ser um dos valores: $\mathcal{O}(\log N)$, $\mathcal{O}(N)$, $\mathcal{O}(N \log N)$ ou $\mathcal{O}(N^2)$ e a coorespondente justificação.

c) Considere um vector A com n inteiros distintos (dimensão de A é $n + 1$). Suponha que a sequência $A[1], A[2], \dots, A[n]$ é unimodal, isto é, os valores crescem até um determinado índice p , entre 0 e n , e depois decrescem até n . $A[p]$ é o valor máximo do vector. Escreva um método em Java que determine o valor máximo do vector em $\mathcal{O}(\log N)$. Justifique a complexidade obtida.

d) Escreva um método recursivo que determine o n -ésimo valor da sequência de quadrados sq_n , com $n \geq 0$, usando a seguinte relação de recorrência:

$$\left. \begin{array}{l} sq_0 = 0, \\ d_0 = 1, \end{array} \right\} \text{ for } n = 0$$
$$\left. \begin{array}{l} sq_n = sq_{n-1} + d_{n-1}, \\ d_n = d_{n-1} + 2, \end{array} \right\} \text{ for } n > 0$$

2. (Valorização: 35%) Classes

Sabendo do seu gosto pelas estruturas de dados, o seu amigo João Atrevido pediu-lhe ajuda para planear uma pequena aplicação de gestão de stocks e pedidos de clientes para os livros que comercializa pela Internet. Essencialmente, a informação a guardar consiste numa lista de clientes, sendo que para cada cliente tem o nome, telefone de contacto e total de livros que adquiriu, e uma segunda lista com as existências de livros que comercializa. Nesta lista temos informação sobre o título do livro, o número de cópias existentes, o número mínimo de cópias a partir do qual é preciso repor o stock, e uma lista de clientes que encomendaram o livro e que será necessário satisfazer.

- a) Escreva uma classe, ou classes, que implementem uma lista ligada genérica, incluindo o(s) método(s) construtores necessários e os métodos para inserir no início da lista e remover o último elemento da lista.
- b) Escreva as classes **Cliente** e **Livro** necessárias para representar objectos com a informação que o seu amigo João pretende guardar. Acrescente ainda ao método **main()** dado uma variável ou mais variáveis que lhe permita guardar efectivamente a informação (i.e. a lista de clientes que alguma vez fizeram uma encomenda, e a lista de livros existentes e pedidos de compra correspondentes).

```
class GerirPedidos {  
    ...  
    public static void main(String args[]) {  
        Scanner kb= new Scanner(System.in);  
        ...  
        // 2b. definir as variáveis com a info sobre clientes e livros  
        ...  
        // input de dados (feito)  
        ...  
        // 2c. escrever tabela de encomendas  
        ...  
    }  
}
```

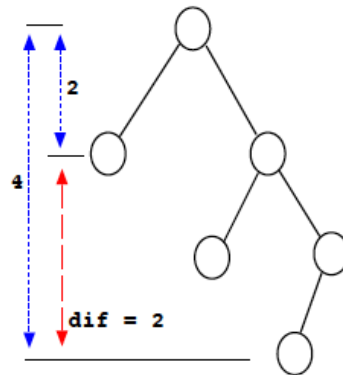
- c) Escreva código, a partir do programa principal, que lhe permita imprimir uma tabela com o número de livros pedidos por cada cliente e, no final, com o total de livros que estão encomendados. A tabela de encomendas a escrever deverá ser algo como:

```
Lista de encomendas:  
Cliente-1  5  
Cliente-2  3  
...  
Total de encomendas: 46
```

Sugestão: defina e use um método que verifique se um dado cliente pertence à lista de encomendas de um dado livro.

3. (Valorização: 35%) Árvores

- a) Defina árvore binária de pesquisa e escreva uma classe ou classes que implementem uma árvore binária de pesquisa em que cada nó da árvore além das referências para os nós filho contém pelo menos dois atributos, uma string que representa a chave e um outro valor inteiro. Inclua os métodos construtores das classes que definir.
- b) Escreva um método que insira um novo nó na árvore, sendo dados como parâmetros os atributos necessários do novo nó.
- c) Escreva um método que determine o factor de “desequilíbrio” da árvore. Este valor corresponde à diferença em valor absoluto entre a altura da folha mais profunda e da menos profunda da árvore. Por exemplo, para a árvore da figura, o valor será 2.



- d) Escreva um método que percorra a árvore em largura e escreva a soma dos valores em cada nível da árvore, com o formato **n:soma**, em que **n** é o nível e **soma** o valor da soma nesse nível. No final escreva a soma total dos valores. Exemplo de um *output* possível para uma dada árvore com altura máxima 3:

```
0:5
1:12
2:34
Total = 51
```